

USAGE AND COSTS

Manage costs effectively

Track token usage, set team spend limits, and reduce Claude Code costs with context management, model selection, extended thinking settings, and preprocessing hooks.

Claude Code charges by API token consumption. For subscription plan pricing (Pro, Max, Team, Enterprise), see claude.com/pricing. Per-developer costs vary widely based on model selection, codebase size, and usage patterns such as running multiple instances or automation.

Across enterprise deployments, the average cost is around \$13 per developer per active day and \$150-250 per developer per month, with costs remaining below \$30 per active day for 90% of users. To estimate spend for your own team, start with a small pilot group and use the tracking tools below to establish a baseline before wider rollout.

This page covers how to [track your costs](#), [manage costs for teams](#), and [reduce token usage](#).

Track your costs

Using the `/usage` command

❗ The Session block in `/usage` shows API token usage and is intended for API users. Claude Max and Pro subscribers have usage included in their subscription, so the session cost figure isn't relevant for billing purposes. Subscribers see plan usage bars, activity stats, and a usage breakdown on the same screen.

The Session block at the top of `/usage` shows detailed token usage statistics for your current session. The dollar figure is an estimate computed locally from token counts and may differ from your actual bill. For authoritative billing, see the Usage page in the [Claude Console](#).

```
Total cost:                $0.55
Total duration (API):      6m 19.7s
Total duration (wall):     6h 33m 10.2s
Total code changes:        0 lines added, 0 lines removed
```

On a Pro, Max, Team, or Enterprise plan, `/usage` also shows a breakdown of what counts against your plan limits. It attributes recent usage to skills, subagents, plugins, and individual MCP servers, with each shown as a percentage of the total. Press `d` or `w` to switch between the last 24 hours and the last 7 days. The figures are approximate and computed from local session history on this machine, so usage from other devices or `claude.ai` is not included.

In the **VS Code extension**, the same breakdown appears in the Account & usage dialog with a Day and Week toggle. Requires Claude Code v2.1.174 or later.

Managing costs for teams

When using Claude API, you can **set workspace spend limits** on the total Claude Code workspace spend. Admins can **view cost and usage reporting** in the Console.

On Pro and Max plans, you can set a monthly spend limit on usage credits with the `/usage-credits` command. If you reach that limit while you still have usage credits available, Claude Code prompts you to raise or remove the limit so you can continue without leaving the CLI. Changing the limit requires billing access on the account.

 When you first authenticate Claude Code with your Claude Console account, a workspace called “Claude Code” is automatically created for you. This workspace provides centralized cost tracking and management for all Claude Code usage in your organization. You cannot create API keys for this workspace; it is exclusively for Claude Code authentication and usage.

For organizations with custom rate limits, Claude Code traffic in this workspace counts toward your organization’s overall API rate limits. You can set a **workspace rate limit** on this workspace’s Limits page in the Claude Console to cap Claude Code’s share and protect other production workloads.

On Bedrock, Vertex, and Foundry, Claude Code does not send metrics from your cloud. Organizations that already route Claude Code through an **LLM gateway** can track spend there, since the gateway sees every request.

Rate limit recommendations

When setting up Claude Code for teams, consider these Token Per Minute (TPM) and Request Per Minute (RPM) per-user recommendations based on your organization size:

Team size	TPM per user	RPM per user
1-5 users	200k-300k	5-7

Team size	TPM per user	RPM per user
5-20 users	100k-150k	2.5-3.5
20-50 users	50k-75k	1.25-1.75
50-100 users	25k-35k	0.62-0.87
100-500 users	15k-20k	0.37-0.47
500+ users	10k-15k	0.25-0.35

For example, if you have 200 users, you might request 20k TPM for each user, or 4 million total TPM (200*20,000 = 4 million).

The TPM per user decreases as team size grows because fewer users tend to use Claude Code concurrently in larger organizations. These rate limits apply at the organization level, not per individual user, which means individual users can temporarily consume more than their calculated share when others aren't actively using the service.

 If you anticipate scenarios with unusually high concurrent usage (such as live training sessions with large groups), you may need higher TPM allocations per user.

Agent team token costs

Agent teams spawn multiple Claude Code instances, each with its own context window. Token usage scales with the number of active teammates and how long each one runs.

To keep agent team costs manageable:

- Use Sonnet for teammates. It balances capability and cost for coordination tasks.
- Keep teams small. Each teammate runs its own context window, so token usage is roughly proportional to team size.
- Keep spawn prompts focused. Teammates load CLAUDE.md, MCP servers, and skills automatically, but everything in the spawn prompt adds to their context from the start.
- Shut down teammates when their work is done. Each active teammate continues consuming tokens until it exits or the session ends.
- Agent teams are disabled by default. Set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in your settings.json or environment to enable them. See enable agent teams.

Reduce token usage

Token costs scale with context size: the more context Claude processes, the more tokens you use. Claude Code automatically optimizes costs through **prompt caching**, which reduces costs for repeated content like system prompts, and auto-compaction, which summarizes conversation history when approaching context limits.

The following strategies help you keep context small and reduce per-message costs.

Manage context proactively

Use `/usage` to check your current token usage, or **configure your status line** to display it continuously.

- **Clear between tasks:** Use `/clear` to start fresh when switching to unrelated work. Stale context wastes tokens on every subsequent message. Use `/rename` before clearing so you can easily find the session later, then `/resume` to return to it.
- **Add custom compaction instructions:** `/compact Focus on code samples and API usage` tells Claude what to preserve during summarization.

You can also customize compaction behavior in your CLAUDE.md:

```
# Compact instructions
```

```
When you are using compact, please focus on test output and code changes
```

Choose the right model

Sonnet handles most coding tasks well and costs less than Opus. Reserve Opus for complex architectural decisions or multi-step reasoning. Use `/model` to switch models mid-session, or set a default in `/config`. For simple subagent tasks, specify `model: haiku` in your **subagent configuration**.

Reduce MCP server overhead

MCP tool definitions are **deferred by default**, so only tool names enter context until Claude uses a specific tool. Run `/context` to see what's consuming space.

- **Prefer CLI tools when available:** Tools like `gh` , `aws` , `gcloud` , and `sentry-cli` are still more context-efficient than MCP servers because they don't add any per-tool listing. Claude can run CLI commands directly.
- **Disable unused servers:** Run `/mcp` to see configured servers and disable any you're not actively using.

Install code intelligence plugins for typed languages

Code intelligence plugins give Claude precise symbol navigation instead of text-based search, reducing unnecessary file reads when exploring unfamiliar code. A single “go to definition” call replaces what might otherwise be a `grep` followed by reading multiple candidate files. Installed language servers also report type errors automatically after edits, so Claude catches mistakes without running a compiler.

Offload processing to hooks and skills

Custom **hooks** can preprocess data before Claude sees it. Instead of Claude reading a 10,000-line log file to find errors, a hook can `grep` for `ERROR` and return only matching lines, reducing context from tens of thousands of tokens to hundreds.

A **skill** can give Claude domain knowledge so it doesn't have to explore. For example, a “codebase-overview” skill could describe your project's architecture, key directories, and naming conventions. When Claude invokes the skill, it gets this context immediately instead of spending tokens reading multiple files to understand the structure.

For example, this `PreToolUse` hook filters test output to show only failures:

settings.json

Add this to your **settings.json** to run the hook before every Bash command:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "~/.claude/hooks/filter-test-output.sh"
          }
        ]
      }
    ]
  }
}
```

filter-test-output.sh

The hook calls this script, which checks if the command is a test runner and modifies it to show only failures:

```
#!/bin/bash
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')

# If running tests, filter to show only failures
if [[ "$cmd" =~ ^(npm test|pytest|go test) ]]; then
  filtered_cmd="$cmd 2>&1 | grep -A 5 -E '(FAIL|ERROR|error:)' |
head -100"
  echo "{\"hookSpecificOutput\":
{\"hookEventName\":\"PreToolUse\",\"permissionDecision\":\"allow\
\",\"updatedInput\":{\"command\":\"$filtered_cmd\"}}}"
else
  echo "{}"
fi
```

Move instructions from CLAUDE.md to skills

Your **CLAUDE.md** file is loaded into context at session start. If it contains detailed instructions for specific workflows (like PR reviews or database migrations), those tokens are present even when you're doing unrelated work. **Skills** load on-demand only when invoked, so moving specialized instructions into skills keeps your base context smaller. Aim to keep CLAUDE.md under 200 lines by including only essentials.

Adjust extended thinking

Extended thinking is enabled by default because it significantly improves performance on complex planning and reasoning tasks. Thinking tokens are billed as output tokens, and the default budget can be tens of thousands of tokens per request depending on the model. For simpler tasks where deep reasoning isn't needed, you can reduce costs by lowering the **effort level** with `/effort` or in `/model`, disabling thinking in `/config`, or, on models with a **fixed thinking budget**, lowering the budget with `MAX_THINKING_TOKENS=8000`. Adaptive-reasoning models ignore nonzero budgets, so use effort levels there instead. Disabling thinking is not available on Fable 5, which always uses extended thinking.

Delegate verbose operations to subagents

Running tests, fetching documentation, or processing log files can consume significant context. Delegate these to **subagents** so the verbose output stays in the subagent's context while only a summary returns to your main conversation.

Manage agent team costs

Agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode, because each teammate maintains its own context window and runs as a separate Claude instance. Keep team tasks small and self-contained to limit per-teammate token usage. See **agent teams** for details.

Write specific prompts

Vague requests like “improve this codebase” trigger broad scanning. Specific requests like “add input validation to the login function in auth.ts” let Claude work efficiently with minimal file reads.

Work efficiently on complex tasks

For longer or more complex work, these habits help avoid wasted tokens from going down the wrong path:

- **Use plan mode for complex tasks:** Press Shift+Tab to enter plan mode before implementation. Claude explores the codebase and proposes an approach for your approval, preventing expensive re-work when the initial direction is wrong.
- **Course-correct early:** If Claude starts heading the wrong direction, press Escape to stop immediately. Use `/rewind` or double-tap Escape to restore conversation and code to a previous checkpoint.
- **Give verification targets:** Include test cases, paste screenshots, or define expected output in your prompt. When Claude can verify its own work, it catches issues before you need to request fixes.
- **Test incrementally:** Write one file, test it, then continue. This catches issues early when they're cheap to fix.

Background token usage

Claude Code uses tokens for some background functionality even when idle:

- **Conversation summarization:** Background jobs that summarize previous conversations for the `claude --resume` feature
- **Command processing:** Some commands like `/usage` may generate requests to check status

These background processes consume a small amount of tokens (typically under \$0.04 per session) even without active interaction.

Understanding changes in Claude Code behavior

Claude Code regularly receives updates that may change how features work, including cost reporting. Run `claude --version` to check your current version. For specific billing questions, contact Anthropic support through your Console account.